Weight-space gradient descent induces approximate activity descent, making learning rules testable

GitHub · Interactive Demo

The problem: synaptic learning rules are hidden, but activity changes are not. A central question in neuroscience is whether biological neural networks optimise a loss function by adjusting synaptic weights—and if so, by what rule. Decades of theoretical work have proposed candidate learning rules. But synaptic weights cannot be measured at scale in behaving animals, so these proposals remain difficult to test directly. Neural *activities*, by contrast, can be recorded in large populations across learning. This raises a concrete question: if the brain does perform gradient descent on its weights, what should we expect to see in the activities? Here we show that the answer takes a surprisingly simple form—activity changes must be approximately proportional to activity-space loss gradients—and that this prediction is directly testable with existing experimental techniques.

Setup and derivation. Consider a feedforward network with layers $\ell = 1, \ldots, L$, weights $\mathbf{W}$, and neural activities $A_i^{(\ell)}(\mathbf{W})$, trained to minimise a loss L. Gradient descent on the weights embodies a simple principle: *the more that strengthening a synapse would improve performance, the more that synapse is strengthened* (and vice versa—harmful synapses are weakened). Formally, every weight is updated simultaneously:

$$\Delta W_\alpha = -\eta \, \frac{\partial L}{\partial W_\alpha}, \qquad \forall\, \alpha. \tag{1}$$

What is the induced change in the activity of neuron i in layer ℓ? Because $A_i^{(\ell)}$ depends only on weights in layers 1 through ℓ, the chain rule gives

$$\Delta A_i^{(\ell)} \;=\; -\eta \sum_{\alpha \in \text{layers } 1..\ell} \frac{\partial A_i^{(\ell)}}{\partial W_\alpha} \frac{\partial L}{\partial W_\alpha}. \tag{2}$$

In a feedforward network, the effect of any weight in layers $1..\ell$ on the loss passes entirely through the activities of layer ℓ, so $\partial L/\partial W_\alpha = \sum_{k \in \text{layer } \ell} (\partial L/\partial A_k^{(\ell)})\,(\partial A_k^{(\ell)}/\partial W_\alpha)$. Exchanging the sums yields

$$\boxed{\Delta A_i^{(\ell)} \;=\; -\eta \sum_{k \in \text{layer } \ell} \Phi_{ik}^{(\ell)} \, \frac{\partial L}{\partial A_k^{(\ell)}},} \tag{3}$$

where

$$\Phi_{ik}^{(\ell)} \;\coloneqq\; \sum_\alpha \frac{\partial A_i^{(\ell)}}{\partial W_\alpha} \frac{\partial A_k^{(\ell)}}{\partial W_\alpha} \;=\; \nabla_{\mathbf{W}} A_i^{(\ell)} \cdot \nabla_{\mathbf{W}} A_k^{(\ell)} \tag{4}$$

is a neural-tangent-kernel-style Gram matrix defined on the activities within each layer, and $\partial L/\partial A_k^{(\ell)}$ is the full backpropagated loss gradient—a sufficient statistic that compresses everything downstream layers contribute to the weight updates affecting layer ℓ. In vector form, $\Delta \mathbf{A}^{(\ell)} = -\eta\, \Phi^{(\ell)} \nabla_{\mathbf{A}^{(\ell)}} L$: gradient descent in weight space induces *kernel descent* within each layer of activity space. This identity is exact and holds for any differentiable feedforward network, regardless of depth, width, or activation function.

The self term drives each neuron toward its own loss gradient. The diagonal element $\Phi_{ii}^{(\ell)} = \|\nabla_{\mathbf{W}} A_i^{(\ell)}\|^2$ collects contributions from *all* weights that influence neuron i—both its own incoming weights and every weight in earlier layers that lies on a path to it. Because $\Phi_{ii}^{(\ell)} > 0$, the self term is always aligned with $-\partial L/\partial A_i^{(\ell)}$: each neuron's activity moves down its own loss gradient, with an effective learning rate $\eta\, \Phi_{ii}^{(\ell)}$ that grows with the depth and width of its input pathways. In other words, weight-space gradient descent guarantees that every neuron has a built-in tendency to change its activity in the loss-reducing direction.

Cross terms introduce interference from other neurons. The off-diagonal elements $\Phi_{ik}^{(\ell)} = \nabla_{\mathbf{W}} A_i^{(\ell)} \cdot \nabla_{\mathbf{W}} A_k^{(\ell)}$ for $k \neq i$ couple neuron i to every other neuron's loss gradient *within the same layer*. Their magnitude reflects parameter sharing: two neurons fed by overlapping early-layer weights will have large $|\Phi_{ik}^{(\ell)}|$; neurons in disjoint pathways will have $\Phi_{ik}^{(\ell)} \approx 0$. These cross terms have no reason to be aligned with $-\partial L/\partial A_i^{(\ell)}$ and constitute interference—the side-effect of the rest of the layer learning simultaneously. Whether the self term or the cross terms dominate determines whether activity changes look like activity-space gradient descent or something messier.

When is the kernel approximately diagonal? The cross terms are small when $\nabla_{\mathbf{W}} A_i^{(\ell)} \cdot \nabla_{\mathbf{W}} A_k^{(\ell)} \approx 0$ for $i \neq k$—that is, when different neurons' activities depend on nearly orthogonal directions in weight space. Whether this occurs depends on the architecture, the parameterization, and the learned representation; it is not guaranteed by width alone.

Width. Increasing width gives each neuron more private incoming parameters, which can enlarge the self term $\Phi_{ii}^{(\ell)}$. But neurons in the same layer also share all upstream parameters, so their activity gradients are generally not independent. The off-diagonal terms therefore need not vanish simply by concentration of measure. In some architectures and regimes, increasing width may still reduce the relative importance of the cross terms, but this is an empirical question rather than a generic theorem.

Depth and parameter sharing. Deeper layers inherit a larger shared upstream parameter set, which can strengthen correlations between neurons' activity gradients. At the same time, each neuron also has its own incoming weights, which contribute a private

component to $\nabla_{\mathbf{W}} A_i^{(\ell)}$. The balance between these shared and private contributions determines whether $\Phi^{(\ell)}$ is close to diagonal, and there is no reason to expect the answer to be universal across depths or architectures.

Learning dynamics. Training may also change this balance. If learning decorrelates representations or routes different neurons through increasingly distinct effective pathways, the off-diagonal entries of $\Phi^{(\ell)}$ may shrink relative to the diagonal. But this link is heuristic and should be treated as a hypothesis to be checked, not assumed.

Taken together, these considerations suggest treating diagonal dominance as a regime that may emerge in some networks, rather than as a general consequence of width. When the diagonal terms dominate the off-diagonal terms, activity changes closely track the negative activity-space loss gradient; when they do not, the full kernel expression in Eq. 3 remains the correct description.

The diagonal approximation: activity descent as a proxy for weight descent. Combining the kernel identity (Eq. 3) with diagonal dominance ($\Phi_{ii}^{(\ell)} \gg \sum_{k\neq i} |\Phi_{ik}^{(\ell)}|$) yields a simple approximation:

$$\Delta A_i^{(\ell)} \;\approx\; -\eta\, \Phi_{ii}^{(\ell)}\, \frac{\partial L}{\partial A_i^{(\ell)}}\,. \tag{5}$$

The intuitive meaning mirrors that of weight descent: just as weight descent says *strengthen each synapse in proportion to how much it helps*, activity descent says *increase each neuron's firing in proportion to how much that increase would help* (and decrease it when it hurts). Equation 5 therefore identifies a useful approximation regime: when $\Phi^{(\ell)}$ is sufficiently diagonal-dominant, each neuron's activity moves in the direction that locally reduces the loss, up to a neuron-specific scaling factor $\Phi_{ii}^{(\ell)}$. The exact identity is always Eq. 3; Eq. 5 is an empirical approximation whose quality depends on the network and training regime.

Recursive structure. The per-layer kernel inherits a recursion from the feedforward computation. Writing $J_\ell = \partial \mathbf{A}^{(\ell)}/\partial \mathbf{A}^{(\ell-1)}$ for the inter-layer Jacobian and $D_\ell = \mathrm{diag}([\sigma'(z_i^{(\ell)})]^2\, \|\mathbf{A}^{(\ell-1)}\|^2)$ for the diagonal kernel from layer ℓ's own incoming weights:

$$\Phi^{(\ell)} = D_\ell + J_\ell\, \Phi^{(\ell-1)}\, J_\ell^T\,, \qquad \Phi^{(1)} = D_1\,. \tag{6}$$

D_ℓ is always diagonal—own-layer weights never couple same-layer neurons. All off-diagonal structure comes from $J_\ell\, \Phi^{(\ell-1)}\, J_\ell^T$: earlier-layer weight updates propagated forward. When the inter-layer Jacobians decorrelate across neurons, this recursion can amplify diagonal structure from earlier layers; when shared upstream structure remains strong, substantial off-diagonal terms can persist.

A testable prediction for neuroscience. This result converts an untestable claim about synaptic learning rules into a testable prediction about observable activity changes. Synaptic weights cannot be measured at scale in vivo, but neural activities can. If a biological circuit performs *any* form of gradient descent on its synaptic weights—even an approximate or noisy variant—and operates in a regime where $\Phi^{(\ell)}$ is approximately diagonal-dominant, then the observable activity changes must satisfy Eq. 5. The per-neuron scaling $\Phi_{ii}^{(\ell)}$ is unknown and may vary across neurons, but the *direction* of $\Delta\mathbf{A}^{(\ell)}$ is pinned: it must be approximately proportional to $-\nabla_{\mathbf{A}^{(\ell)}} L$. A concrete experimental protocol follows: record neural population activity across learning trials, estimate $\partial L/\partial A_i^{(\ell)}$ from a task-fitted model, and check whether the measured $\Delta A_i^{(\ell)}$ is proportional to the predicted gradient. Only the direction matters, not the magnitude—the unknown $\Phi_{ii}^{(\ell)}$ absorbs all per-neuron scaling. A failure of proportionality would rule out not just backpropagation but *any* weight-space gradient rule in a regime where diagonal dominance is expected to hold.

Caveat: activity descent is necessary but not sufficient for weight descent. The converse is weaker: observing that activities follow their loss gradient constrains the projection of $\Delta\mathbf{W}$ onto the row space of the Jacobian $\partial A_i^{(\ell)}/\partial W_\alpha$, but says nothing about the (typically vast) null space of weight changes that leave activities unchanged on the current input. Activity-level gradient descent is therefore consistent with many different weight-level learning rules, of which backpropagation is the minimum-norm solution. In short, weight descent always implies the kernel relation in Eq. 3, and implies the simpler activity-descent form in Eq. 5 only when $\Phi^{(\ell)}$ is close to diagonal; activity descent itself does not uniquely identify the weight-level rule. This asymmetry is a feature, not a bug: it means the test can *falsify* gradient-based learning, even though it cannot confirm a specific synaptic rule.

Methods. Figures 1–2 numerically verify Eqs. 2–5 in a 3-hidden-layer ReLU MLP trained with online SGD ($\eta = 0.005$) on a 2-class image dataset (4×4 pixel images of horizontal vs. vertical bars, 20 per class, input dimension 16, output dimension 2). Dead ReLU neurons (pre-activation ≤ 0 on the sampled input) are excluded from all analyses. At regular intervals during training, we compute the full activity Jacobian $\partial A_i^{(\ell)}/\partial W_\alpha$ and the per-layer kernel $\Phi_{ik}^{(\ell)}$, then compare the predicted activity change ΔA from the chain-rule identity (Eq. 2), the kernel equation (Eq. 3), and the diagonal approximation (Eq. 5) against the actual change measured after a single gradient step. Figure 1 shows detailed diagnostics at widths 8 and 48. Figure 2 sweeps width from 4 to 128 (3 random seeds per width, each trained for 2000 SGD steps) to quantify how the diagonal approximation in Eq. 5 improves with width in this toy setting.

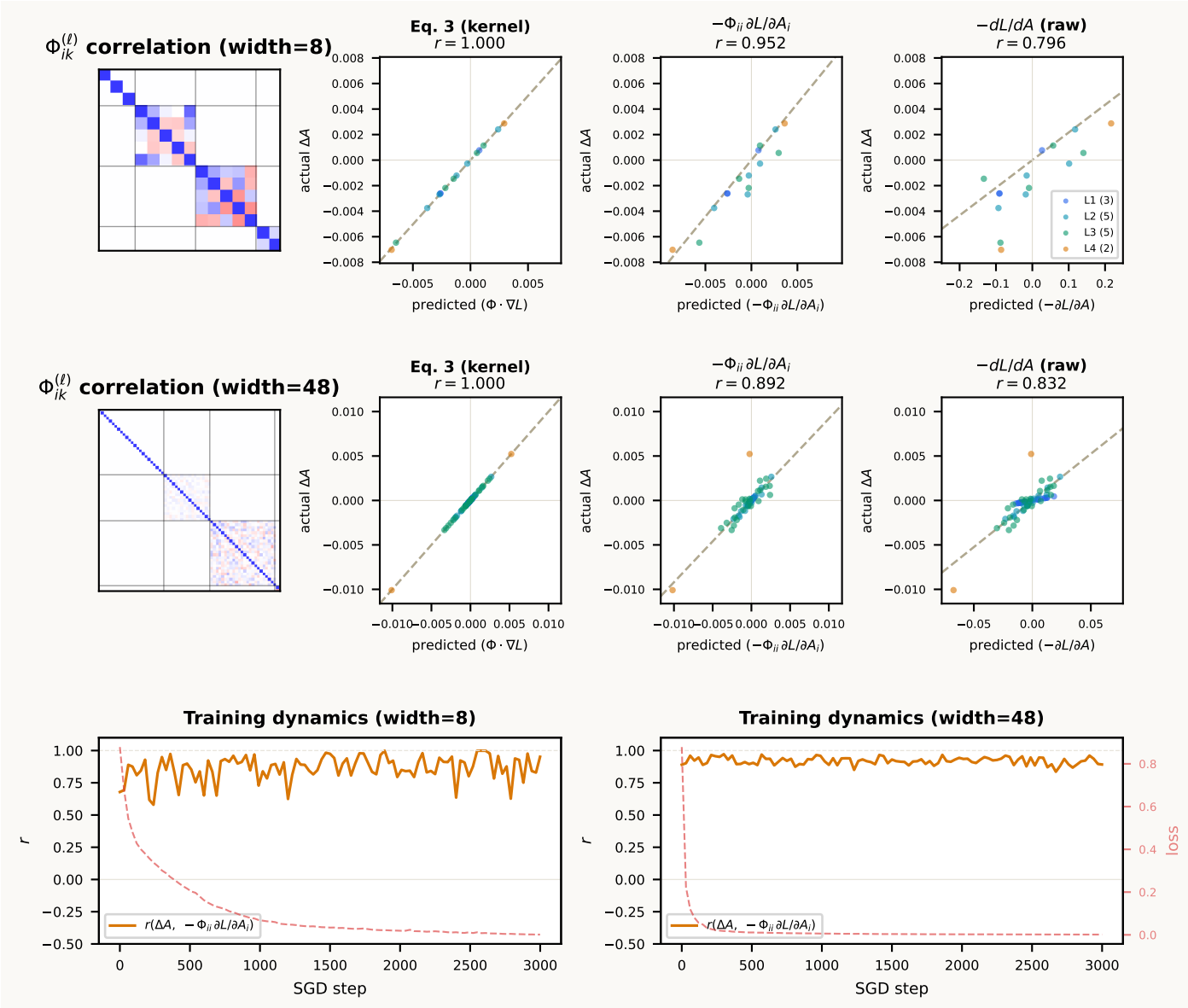

Figure 1: **In a wide network doing weight descent, activity changes closely match activity-space gradient descent.** Top row: width = 8. Middle row: width = 48. Left: normalised $\Phi_{ik}^{(\ell)}$ heatmaps (white = uncorrelated, blue = correlated). Scatter panels: predicted vs. actual ΔA for the full kernel (Eq. 3), the diagonal approximation $-\Phi_{ii}\,\partial L/\partial A_i$, and raw $-\partial L/\partial A$. Dead ReLU neurons excluded. Bottom row: $r(\Delta A,\ -\Phi_{ii}\,\partial L/\partial A_i)$ and loss over training.

A Annotated layer-2 decomposition

To make the diagonal/off-diagonal split concrete, consider a standard layer-2 block (omitting biases for simplicity):

$$z_i^{(2)} = \sum_m W_{im}^{(2)} A_m^{(1)}, \qquad A_i^{(2)} = \sigma_2\big(z_i^{(2)}\big).$$

The exact activity update is

$$\Delta A_i^{(2)} = -\eta \sum_j \Phi_{ij}^{(2)} \frac{\partial L}{\partial A_j^{(2)}}.$$

The point of this appendix is to show explicitly where the self term $\Phi_{ii}^{(2)}$ and the cross terms $\Phi_{ij}^{(2)}$ for $j \neq i$ come from.

Own-layer weights contribute only to the diagonal. For a weight $W_{pm}^{(2)}$ feeding directly into layer 2,

$$\frac{\partial A_i^{(2)}}{\partial W_{pm}^{(2)}} = \delta_{ip}\, \sigma_2'\big(z_i^{(2)}\big)\, A_m^{(1)}.$$

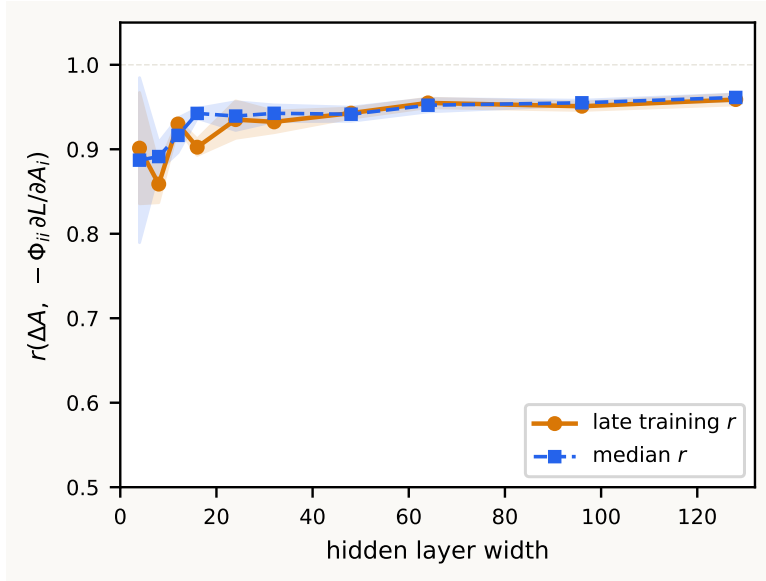

Figure 2: **The diagonal approximation improves with width in this toy setting.** Pearson r between actual ΔA and $-\Phi_{ii}\,\partial L/\partial A_i$ as a function of hidden-layer width. Dead ReLU neurons excluded. Shaded regions: ± 1 s.d. across seeds.

Therefore the layer-2 contribution to the kernel is

$$\sum_{p,m} \frac{\partial A_i^{(2)}}{\partial W_{pm}^{(2)}} \frac{\partial A_j^{(2)}}{\partial W_{pm}^{(2)}} = \delta_{ij}\,[\sigma_2'(z_i^{(2)})]^2\,\|\mathbf{A}^{(1)}\|^2 = \delta_{ij} D_{2,ii}.$$

This term is diagonal because two different neurons in the same layer do not share their own incoming weights.

Earlier-layer weights are propagated forward by the Jacobian. Now take a weight W_α in layer 1. By the chain rule,

$$\frac{\partial A_i^{(2)}}{\partial W_\alpha} = \sum_m \frac{\partial A_i^{(2)}}{\partial A_m^{(1)}} \frac{\partial A_m^{(1)}}{\partial W_\alpha} = \sum_m J_{2,im} \frac{\partial A_m^{(1)}}{\partial W_\alpha}, \qquad J_{2,im} = \frac{\partial A_i^{(2)}}{\partial A_m^{(1)}}.$$

Summing over all layer-1 weights gives

$$\sum_{\alpha \in \text{layer } 1} \frac{\partial A_i^{(2)}}{\partial W_\alpha} \frac{\partial A_j^{(2)}}{\partial W_\alpha} = \sum_{m,n} J_{2,im}\,\Phi_{mn}^{(1)}\,J_{2,jn}.$$

Because layer 1 has no earlier shared weights, $\Phi^{(1)} = D_1$ is diagonal, so this reduces to

$$\sum_{m,n} J_{2,im}\,\Phi_{mn}^{(1)}\,J_{2,jn} = \sum_m J_{2,im}\,D_{1,mm}\,J_{2,jm}.$$

Thus the full layer-2 kernel is

$$\Phi_{ij}^{(2)} = \delta_{ij} D_{2,ii} + \sum_m J_{2,im}\,D_{1,mm}\,J_{2,jm}.$$

If one writes the Jacobian explicitly,

$$J_{2,im} = \sigma_2'(z_i^{(2)}) W_{im}^{(2)},$$

so

$$\Phi_{ij}^{(2)} = \delta_{ij}[\sigma_2'(z_i^{(2)})]^2 \|\mathbf{A}^{(1)}\|^2 + \sigma_2'(z_i^{(2)})\,\sigma_2'(z_j^{(2)}) \sum_m W_{im}^{(2)} D_{1,mm} W_{jm}^{(2)}.$$

Self and cross terms in the activity update. Substituting the layer-2 kernel into the exact update and separating the $j = i$ and $j \neq i$ pieces gives

$$\Delta A_i^{(2)} = -\eta\,\Phi_{ii}^{(2)} \frac{\partial L}{\partial A_i^{(2)}} - \eta \sum_{j \neq i} \Phi_{ij}^{(2)} \frac{\partial L}{\partial A_j^{(2)}}.$$

Using the expression above,

$$\Phi_{ii}^{(2)} = D_{2,ii} + \sum_m J_{2,im}^2 D_{1,mm},$$

while for $j \neq i$,

$$\Phi_{ij}^{(2)} = \sum_m J_{2,im} D_{1,mm} J_{2,jm}.$$

Hence

$$\Delta A_i^{(2)} = -\eta\Big(D_{2,ii} + \sum_m J_{2,im}^2 D_{1,mm}\Big)\frac{\partial L}{\partial A_i^{(2)}} - \eta\sum_{j\neq i}\Big(\sum_m J_{2,im} D_{1,mm} J_{2,jm}\Big)\frac{\partial L}{\partial A_j^{(2)}}.$$

The first bracket is the self term: it is always aligned with $-\partial L/\partial A_i^{(2)}$. The second sum is the interference term: it mixes neuron i with the gradients of other neurons in the same layer through shared earlier-layer parameters.

Relation to the temporary x_0/x_1 notation. If one writes $x_0 = \mathbf{A}^{(0)}$ and $x_1 = \mathbf{A}^{(1)}$, then

$$D_{1,mm} = [\sigma_1'(z_m^{(1)})]^2\|x_0\|^2, \qquad D_{2,ii} = [\sigma_2'(z_i^{(2)})]^2\|x_1\|^2.$$

So the appearance of both x_0 and x_1 in scratch work is meaningful, not a typo: they are simply the activities entering layers 1 and 2. In the main text, $\mathbf{A}^{(0)}$ and $\mathbf{A}^{(1)}$ are cleaner and more consistent with the rest of the notation.

B General recursion for deeper networks and nonlinearities

For an arbitrary depth-L feedforward network with pointwise nonlinearities,

$$z_i^{(\ell)} = \sum_m W_{im}^{(\ell)} A_m^{(\ell-1)}, \qquad A_i^{(\ell)} = \sigma_\ell(z_i^{(\ell)}), \qquad \ell = 1,\dots,L,$$

again omitting biases for simplicity. The exact kernel identity from the main text still holds:

$$\Delta\mathbf{A}^{(\ell)} = -\eta\,\Phi^{(\ell)}\,\nabla_{\mathbf{A}^{(\ell)}} L, \qquad \Phi_{ij}^{(\ell)} = \sum_\alpha \frac{\partial A_i^{(\ell)}}{\partial W_\alpha}\frac{\partial A_j^{(\ell)}}{\partial W_\alpha}.$$

To obtain a recursion, split the weight sum into layer ℓ itself and layers $1,\dots,\ell-1$.

Own-layer contribution. For a weight $W_{pm}^{(\ell)}$ in layer ℓ,

$$\frac{\partial A_i^{(\ell)}}{\partial W_{pm}^{(\ell)}} = \delta_{ip}\,\sigma_\ell'(z_i^{(\ell)})\,A_m^{(\ell-1)}.$$

Therefore the own-layer contribution is diagonal:

$$\sum_{p,m}\frac{\partial A_i^{(\ell)}}{\partial W_{pm}^{(\ell)}}\frac{\partial A_j^{(\ell)}}{\partial W_{pm}^{(\ell)}} = \delta_{ij}[\sigma_\ell'(z_i^{(\ell)})]^2\|\mathbf{A}^{(\ell-1)}\|^2 = \delta_{ij}D_{\ell,ii}.$$

Earlier-layer contribution. Define the inter-layer Jacobian

$$J_{\ell,im} = \frac{\partial A_i^{(\ell)}}{\partial A_m^{(\ell-1)}} = \sigma_\ell'(z_i^{(\ell)})W_{im}^{(\ell)}.$$

Then for any weight W_α in layers $1,\dots,\ell-1$,

$$\frac{\partial A_i^{(\ell)}}{\partial W_\alpha} = \sum_m J_{\ell,im}\,\frac{\partial A_m^{(\ell-1)}}{\partial W_\alpha}.$$

Summing over earlier-layer weights gives

$$\sum_{\alpha\in\text{layers }1..\ell-1}\frac{\partial A_i^{(\ell)}}{\partial W_\alpha}\frac{\partial A_j^{(\ell)}}{\partial W_\alpha} = \sum_{m,n} J_{\ell,im}\,\Phi_{mn}^{(\ell-1)}\,J_{\ell,jn}.$$

Combining both pieces yields the recursion

$$\Phi^{(\ell)} = D_\ell + J_\ell\,\Phi^{(\ell-1)}\,J_\ell^T, \qquad \Phi^{(1)} = D_1.$$

This is exactly Eq. (6) in expanded form.

Unrolling the recursion. Repeated substitution shows that every earlier layer contributes a positive-semidefinite term transported forward by the intermediate Jacobians:

$$\Phi^{(\ell)} = D_\ell + J_\ell D_{\ell-1}J_\ell^T + J_\ell J_{\ell-1}D_{\ell-2}J_{\ell-1}^T J_\ell^T + \cdots + J_\ell J_{\ell-1}\cdots J_2\,D_1\,J_2^T\cdots J_{\ell-1}^T J_\ell^T.$$

This makes the structure transparent: D_r captures the direct effect of layer-r weights on layer-r activities, and the surrounding Jacobians propagate that contribution forward to layer ℓ.

Role of nonlinearities. Nothing in the exact identity requires linear hidden units. For smooth nonlinearities such as sigmoid, tanh, or GELU, the derivation above is exact with the corresponding σ'_ℓ. For piecewise-linear nonlinearities such as ReLU or leaky ReLU, the same formulas hold almost everywhere; at kink points one may use a subgradient. In particular, if a ReLU unit is inactive on the sampled input, then $\sigma'_\ell\big(z_i^{(\ell)}\big) = 0$, so its row of J_ℓ and its diagonal entry in D_ℓ vanish. This is the appendix-level version of why dead ReLU units contribute nothing to the kernel on that input.

More general feedforward modules. The recursion above assumes an affine layer followed by a pointwise nonlinearity. The exact kernel identity

$$\Phi_{ij}^{(\ell)} = \sum_\alpha \frac{\partial A_i^{(\ell)}}{\partial W_\alpha} \frac{\partial A_j^{(\ell)}}{\partial W_\alpha}$$

still holds for any differentiable feedforward architecture. For other modules the same logic applies: there is an own-module contribution from parameters in layer ℓ and a propagated earlier-layer contribution obtained by sandwiching $\Phi^{(\ell-1)}$ between the appropriate Jacobians. Only the explicit forms of D_ℓ and J_ℓ change.